Experiment – 3

Development of an Interactive Event Page using React Hooks and State Management

---

Name : Shivam Sandeep Kshirsagar

Class : D15A

Roll No. : 23

Date : 22/10/26

Aim

Develop an using **React Hooks and State Management**, and to understand the use of useState, useEffect, and useContext for managing component state, handling side effects, and sharing data between React components.

Problem Statement

Develop an interactive Event Management Page using React.

Enhance the Event Page developed in the previous experiment by adding interactive features such as **event search, category filtering, registration status, and shared application state**.

The application should demonstrate the use of React Hooks for managing and updating data dynamically.

The following React concepts should be demonstrated:

- *useState* for managing local component state
- *useEffect* for handling side effects
- *useContext* for sharing state between components
- State management using Context API
- Event handling in React
- Conditional rendering
- Passing and updating state between components
- Dynamic rendering of event information

Or the same can be demonstrated on your project topic.

Theory Questions

1. What are React Hooks?
Ans: React Hooks are built-in functions that allow developers to "hook into" React state and lifecycle features from functional components, eliminating the need to write class components.

2. What is the purpose of useState()?
Ans: The `useState()` hook is used to declare, read, and update local state variables within functional components.

3. What is the difference between a normal variable and state in React?
Ans: When a normal JavaScript variable is updated, it does not affect the UI. When a React state variable is updated, it triggers a component re-render, ensuring the UI accurately reflects the new data.

4. What is useEffect()?
Ans: `useEffect()` is a hook used to handle side effects in functional components, such as data fetching, setting up subscriptions, or manually manipulating the DOM.

5. When is useEffect() executed?
Ans: By default, it executes after the initial render and after every subsequent update. However, its execution frequency is strictly controlled by its dependency array.

6. What is the purpose of the dependency array in useEffect()?
Ans:The dependency array optimizes performance by instructing React to only re-run the effect if the specific variables listed inside the array have changed between renders. If the array is empty ([]), the effect runs only once on the initial mount.

7. What is useContext()?
Ans: `useContext()` is a hook that allows a functional component to consume and read the current value from a React Context directly, without needing to wrap the component in a Context Consumer.

8. What is Context API in React?
Ans: The Context API is a built-in feature designed to share global or application-wide data (like themes, authentication status, or user settings) across the component tree without manually passing props.

9. What is prop drilling?
Ans: Prop drilling is the cumbersome process of passing data via props from a top-level component down to deeply nested child components, routing through intermediate components that do not actually need the data themselves.

10. How does Context API help in avoiding prop drilling?
Ans: It provides a centralized data provider at a high level. Any nested component, regardless of how deep it is in the tree, can directly access that data using `useContext()`, bypassing all intermediate parent components.

11. What is state management?
Ans: State management is the methodology of handling, organizing, and tracking dynamic data (state) across a React application to ensure the user interface stays synchronized with the underlying data.

12. What is the difference between local state and global/shared state?
Ans:

- **Local state:** Confined to a single component (e.g., a toggle switch or a text input field). Managed via `useState()`.
- **Global/shared state:** Needed by multiple components across the application (e.g., a shopping cart or user login token). Managed via Context API or external libraries like Redux.

13. What is conditional rendering in React?
Ans: Conditional rendering is the practice of instructing React to render different UI elements or components based on specific conditions or state values, typically using ternary operators (`condition ? true : false`) or logical AND (`condition && UI`).

14. How does React update the UI when state changes?
Ans: When state changes, React triggers a re-render. It creates a new Virtual DOM, compares it to the previous Virtual DOM (a process called reconciliation), determines the exact differences, and then efficiently updates only those specific changed nodes in the actual browser DOM.

15. Why are Hooks useful in modern React applications?
Ans: Hooks simplify code by allowing developers to manage state and side effects using functional components rather than complex class components. They promote modularity, make it easier to extract and reuse stateful logic across different components, and eliminate issues with `this` binding.

Code

1. useState() for Managing Local Component State

```
const [searchTerm, setSearchTerm] =
  useState('');

const [selectedCategory, setSelectedCategory] =
  useState('All');

const [selectedType, setSelectedType] =
  useState('All');

const [sortBy, setSortBy] =
  useState('recent');

const [applicationMessage, setApplicationMessage] =
  useState('');
```

2. useEffect() for Handling Side Effects

```
useEffect(() => {

  document.title = searchTerm
    ? `Searching "${searchTerm}" | JobConnect`
    : 'JobConnect | Find Your Next Opportunity';

}, [searchTerm]);
```

```
useEffect(() => {

  if (!applicationMessage) {
    return;
  }

  const timer = setTimeout(() => {
    setApplicationMessage('');
  }, 3000);

  return () => clearTimeout(timer);

}, [applicationMessage]);
```

3. createContext() and Context API

```
const JobContext = createContext(null);

export const JobProvider = ({ children }) => {
  // Shared application state
  const [registeredJobs, setRegisteredJobs] = useState(() => {
    const savedJobs = localStorage.getItem('jobconnect_registered_jobs');

    return savedJobs ? JSON.parse(savedJobs) : [];
  });
```

```
return (
  <JobContext.Provider
    value={{
      registeredJobs,
      registerForJob,
      isRegistered,
    }}
  >
    {children}
  </JobContext.Provider>
);
```

4. useContext() for Sharing State Between Components

```
export const useJobs = () => {
  const context = useContext(JobContext);

  if (!context) {
    throw new Error(
      'useJobs must be used inside JobProvider'
    );
  }

  return context;
};
```

5. State Update and Job Registration

```
const registerForJob = (job) => {
  setRegisteredJobs((currentJobs) => {
    // Prevent duplicate registration
    if (
      currentJobs.some(
        (registeredJob) => registeredJob.id === job.id
      )
    ) {
      return currentJobs;
    }

    return [...currentJobs, job];
  });
};
```

```
const handleRegistration = () => {
  if (!registered) {
    registerForJob(job);

    if (onRegistered) {
      onRegistered(job.title);
    }
  }
};
```

6. Search and Category Filtering

```
const filteredJobs = useMemo(() => {

  const normalizedSearch =
    searchTerm.trim().toLowerCase();

  const result = jobs.filter((job) => {

    const searchableText = [
      job.title,
      job.company,
      job.location,
      job.category,
      ...job.skills,
    ]
      .join(' ')
      .toLowerCase();

    const matchesSearch =
      normalizedSearch === '' ||
      searchableText.includes(
        normalizedSearch
      );

    const matchesCategory =
      selectedCategory === 'All' ||
      job.category === selectedCategory;

    const matchesType =
      selectedType === 'All' ||
      job.type === selectedType;

    return (
      matchesSearch &&
      matchesCategory &&
      matchesType
    );
  });
```

7. Event Handling and Conditional Rendering

```
<input
  type="text"
  value={searchTerm}
  onChange={(event) =>
    setSearchTerm(
      event.target.value
    )
  }
  placeholder="Search jobs, companies, skills or locations..."
  className="h-12 w-full rounded-xl border border-slate-700 bg-slate-800 pl-12 pr-4 text-sm text-white outline-none placeholder:text-slate-500
  focus:border-blue-500 focus:ring-2 focus:ring-blue-500/20"
/>
```

```
{registeredJobs.length > 0 ? (

  <div className="grid gap-3 p-6 md:grid-cols-2">

    {registeredJobs.map(
      (job) => (

        <div
          key={job.id}
          className="flex items-center gap-4 rounded-xl border border-slate-200 p-4"
        >

          <div className="grid h-11 w-11 shrink-0 place-items-center rounded-xl bg-slate-900 text-sm font-black text-white">
            {job.logo}
          </div>

          <div className="min-w-0">

            <p className="truncate font-bold text-slate-800">
              {job.title}
            </p>

            <p className="text-sm text-slate-500">
              {job.company}
              {' · '}
              {job.location}
            </p>

          </div>

          <span className="material-symbols-outlined ml-auto text-emerald-500">
            check_circle
          </span>

        </div>

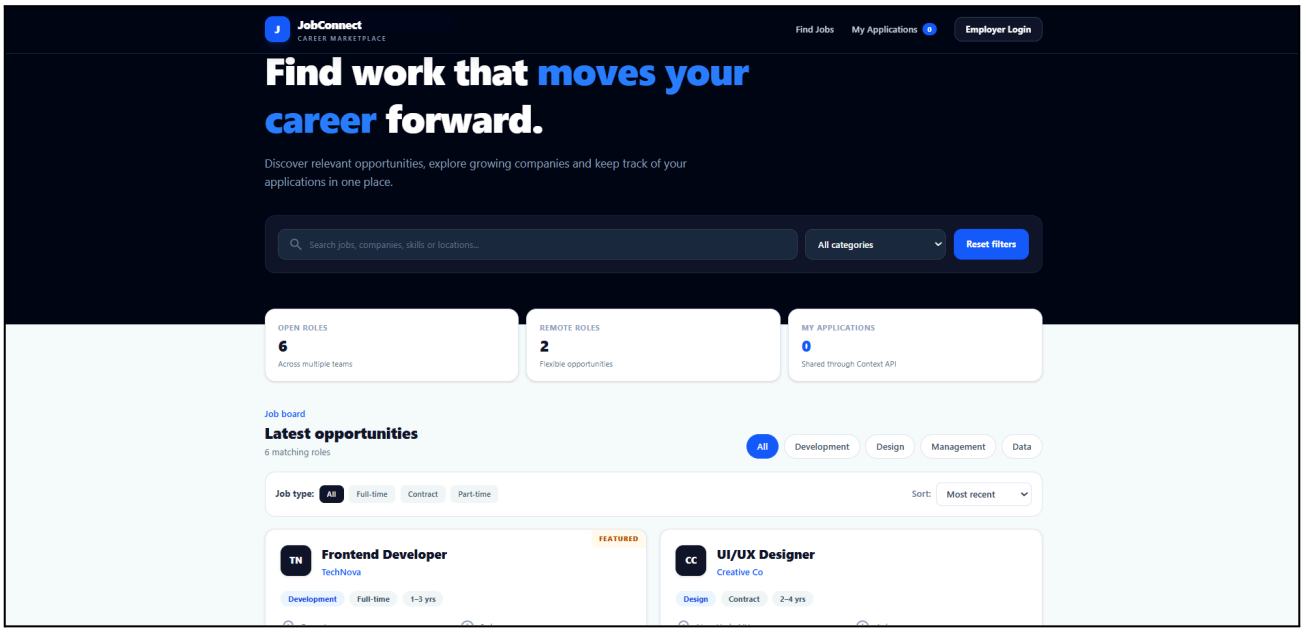
      )
    )}

  </div>

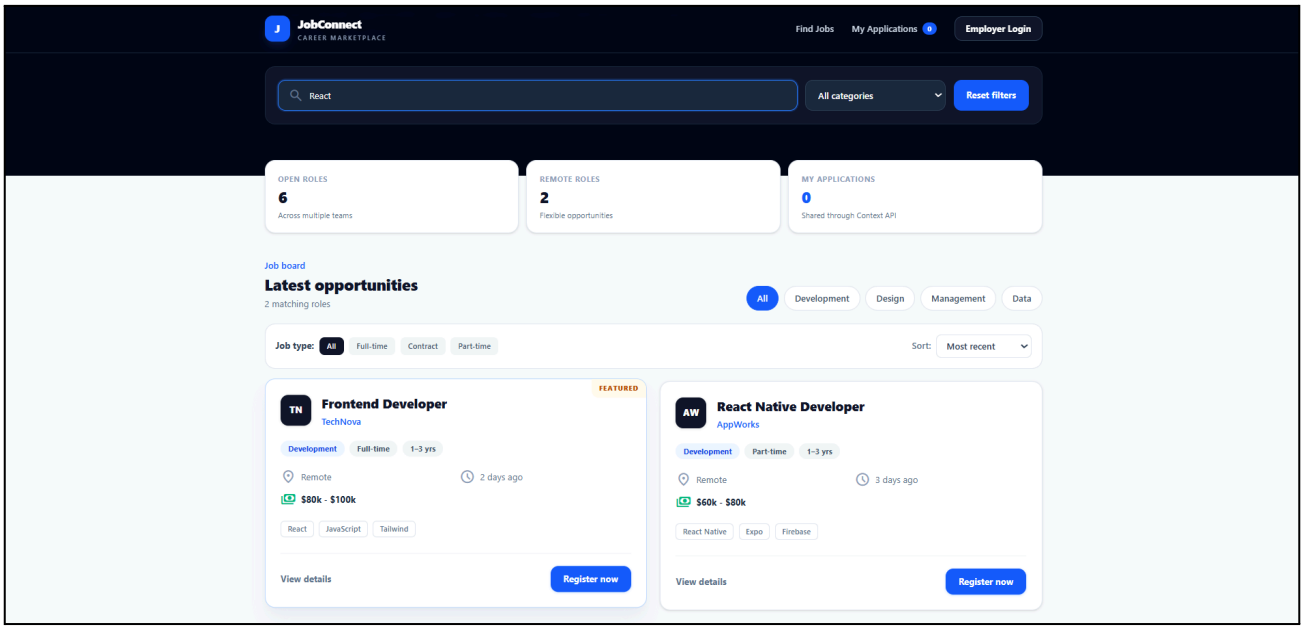
) : (
```

OUTPUT

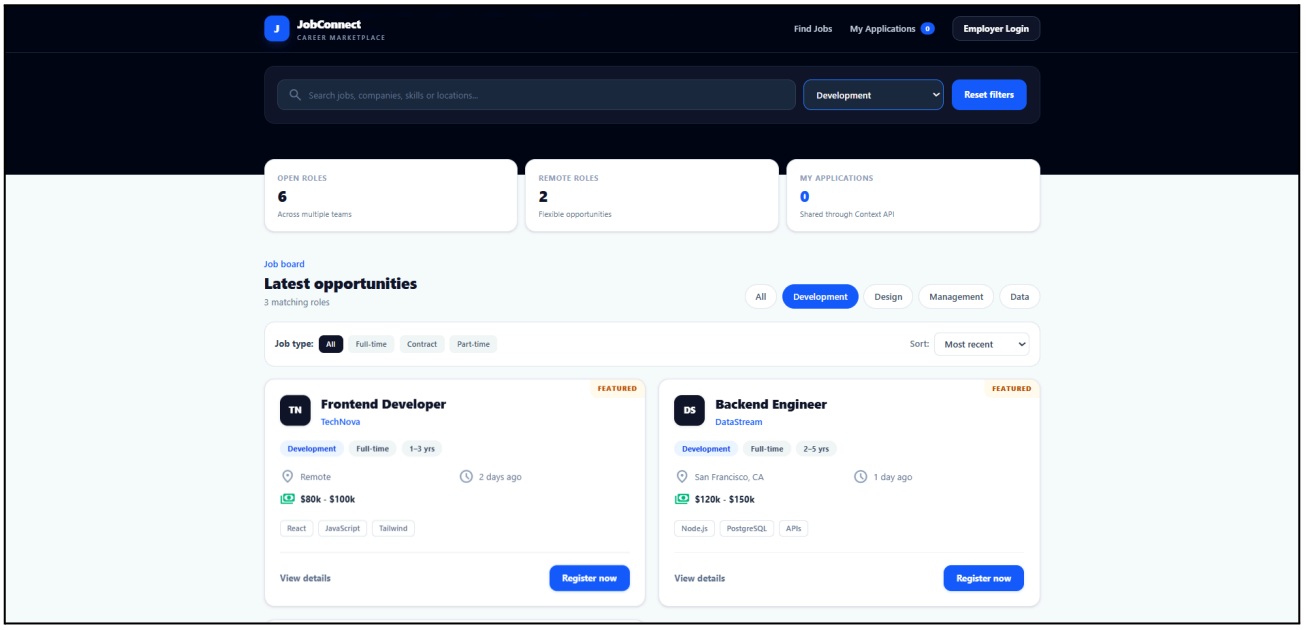
1. Job Portal Home Page with Dynamic Job Listings



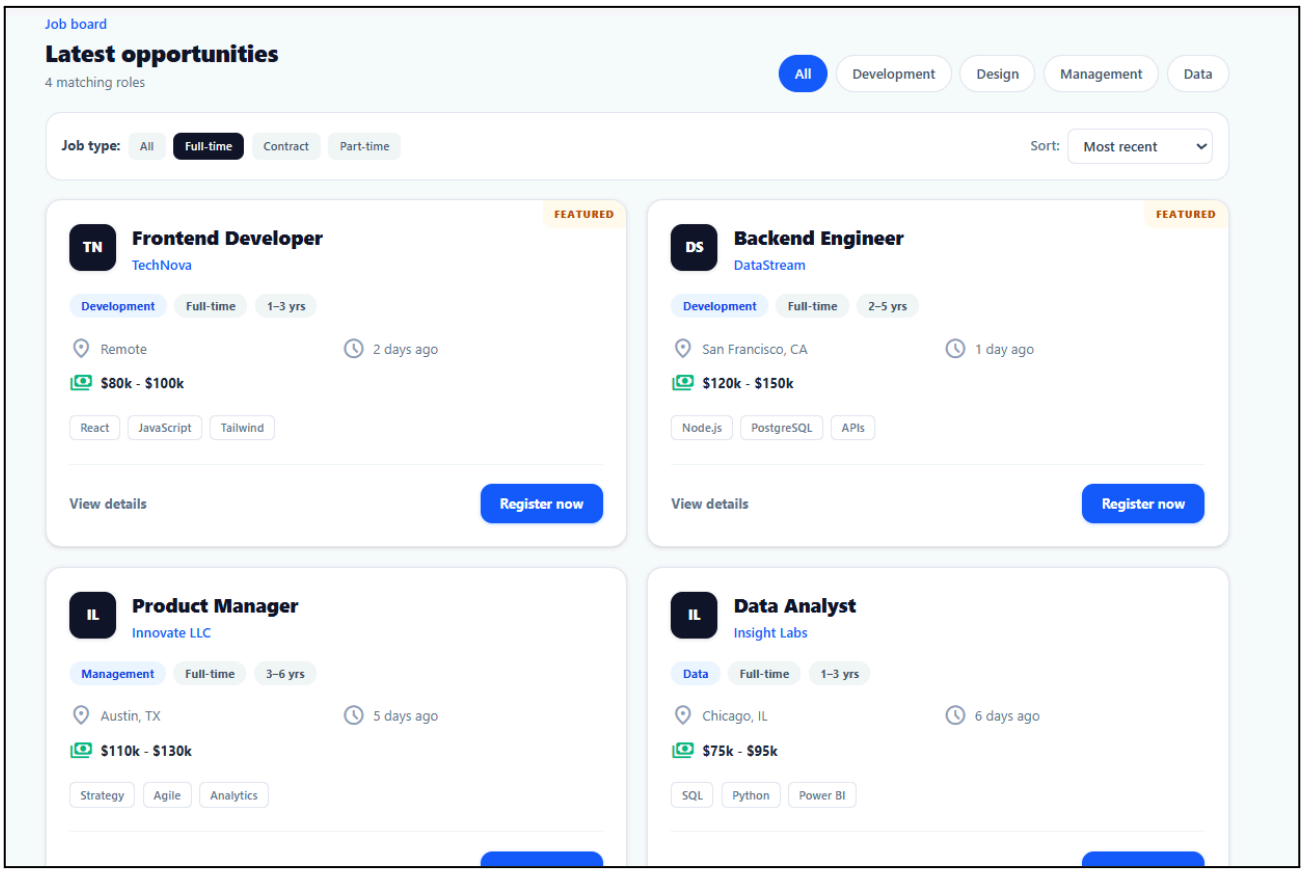
2. Job Search Using Search Bar



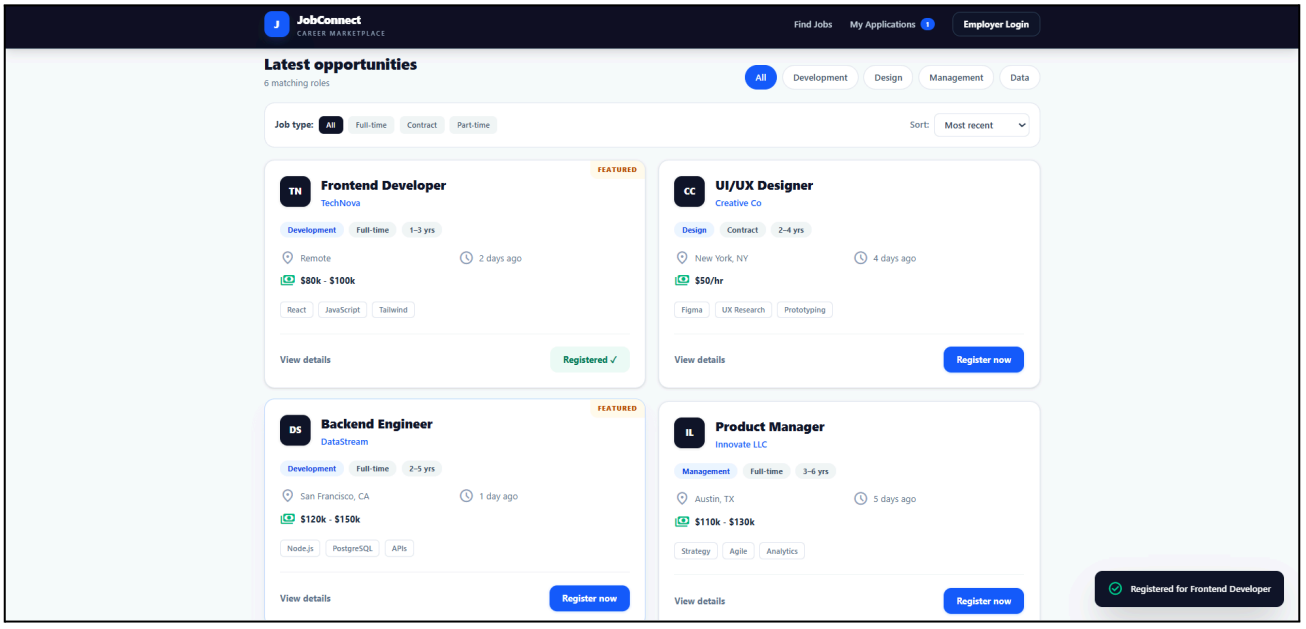
3. Job Category Filtering



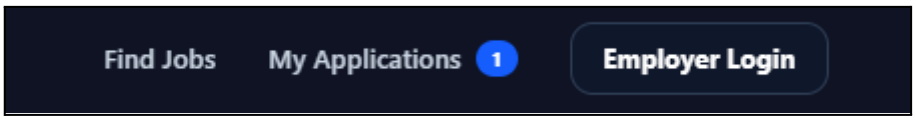
4. Job Type Filtering

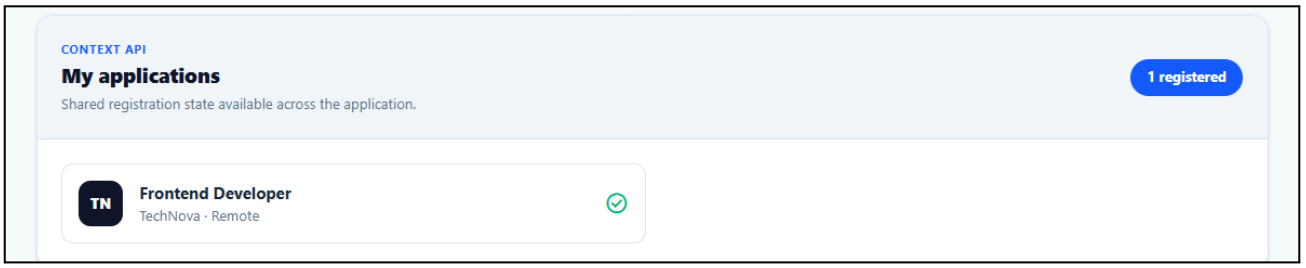


5. Job Registration and Registration Status

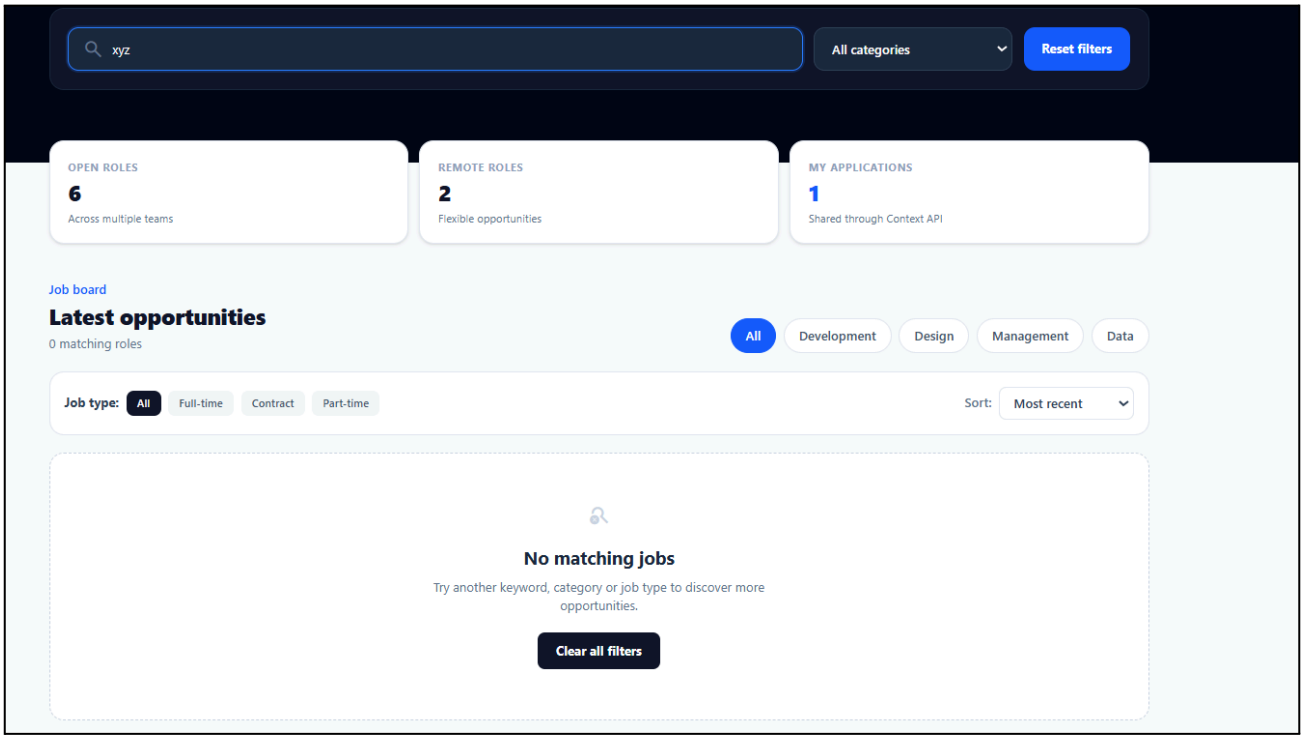


6. Shared Application State using Context API





7. Conditional Rendering for No Matching Jobs



Repository Link: https://github.com/ShivammmKshirsagar/Job-Portal.git